Master 1 informatique - SPP - Travaux dirigés
TD 5 : logique dynamique

Alain Giorgetti

Ce document est protégé par des droits d'auteur et ne peut être traité, reproduit ou partagé par des systèmes automatisés sans autorisation explicite de l'auteur.

Utiliser la logique dynamique avec substitutions explicites pour prouver les formules données, en suivant les étapes indiquées dans les questions. Les règles de raisonnement sont rappelées à la fin de ce document (ces règles sont expliquées dans les cours intitulés "Java et la logique dynamique" et "Le prouveur KeY et la logique du premier ordre").

1 Carré

On considère la formule de logique dynamique reproduite dans la figure 1.

```
\programVariables {
 int i;
 int x;
 int y;
}

\problem {
  (x >= 0 & wellFormed(heap)) ->
  \[
    {
      i = 0;
      y = 0;
      while (i < x) {
        y = y + 2 * i + 1;
        i = i + 1;
      }
    }
 \] (y = x * x)
}
```

FIGURE 1 – Calcul itératif du carré d'un entier.

1. Recopier et compléter la première étape suivante de la preuve formelle de cette formule :

$$\frac{\ldots \Longrightarrow \ldots}{\Longrightarrow (x \geq 0) \rightarrow [\{i = 0; y = 0; \textbf{while } (i < x)\{y = y + 2 * i + 1; i = i + 1;\}\}](y = x * x)} \text{ (impRight)}$$

2. Présenter de même les étapes suivantes du raisonnement, jusqu'à obtenir une instruction **while** entre crochets.
3. Proposer un invariant pour la boucle de ce programme.
4. Appliquer la règle (*tant-que*) avec cet invariant.
5. Prouver séparément les trois prémisses de cette règle (invariant initialement valide, préservation de l'invariant, l'invariant permet de prouver la postcondition), en appliquant les substitutions, en utilisant le calcul des séquents de la logique du premier ordre, et le calcul arithmétique usuel.

2 Racine carrée

On considère la formule de logique dynamique reproduite dans la figure 2.

```
\programVariables {
  int n;
  int r;
}

\problem {
  n >= 0 & wellFormed(heap) ->
  \[
    {
     r = n;
     while (r * r > n)
       r = r - 1;
    }
  \] (r * r <= n & (r+1)*(r+1) > n)
}
```

FIGURE 2 – Calcul de la racine carrée par défaut.

1. Présenter une par une les premières étapes du raisonnement, jusqu'à obtenir une instruction **while** entre crochets.
2. Proposer un invariant pour la boucle de ce programme.
3. Avec cet invariant, appliquer la règle (*tant-que*) au séquent obtenu dans la question 1.
4. Prouver formellement, séparément, les trois prémisses de cette règle, en utilisant les règles de la logique dynamique avec substitutions explicites et du calcul des séquents de la logique du premier ordre.

"Propositional" Sequent Calculus Rules

main	left side (antecedent)	right side (succedent)
not	$\dfrac{\Gamma \Longrightarrow \phi, \Delta}{\Gamma, \neg\phi \Longrightarrow \Delta}$	$\dfrac{\Gamma, \phi \Longrightarrow \Delta}{\Gamma \Longrightarrow \neg\phi, \Delta}$
and	$\dfrac{\Gamma, \phi, \psi \Longrightarrow \Delta}{\Gamma, \phi \wedge \psi \Longrightarrow \Delta}$	$\dfrac{\Gamma \Longrightarrow \phi, \Delta \qquad \Gamma \Longrightarrow \psi, \Delta}{\Gamma \Longrightarrow \phi \wedge \psi, \Delta}$
or	$\dfrac{\Gamma, \phi \Longrightarrow \Delta \qquad \Gamma, \psi \Longrightarrow \Delta}{\Gamma, \phi \vee \psi \Longrightarrow \Delta}$	$\dfrac{\Gamma \Longrightarrow \phi, \psi, \Delta}{\Gamma \Longrightarrow \phi \vee \psi, \Delta}$
imp	$\dfrac{\Gamma \Longrightarrow \phi, \Delta \qquad \Gamma, \psi \Longrightarrow \Delta}{\Gamma, \phi \rightarrow \psi \Longrightarrow \Delta}$	$\dfrac{\Gamma, \phi \Longrightarrow \psi, \Delta}{\Gamma \Longrightarrow \phi \rightarrow \psi, \Delta}$

close $\dfrac{}{\Gamma, \phi \Longrightarrow \phi, \Delta}$ true $\dfrac{}{\Gamma \Longrightarrow \mathrm{true}, \Delta}$ false $\dfrac{}{\Gamma, \mathrm{false} \Longrightarrow \Delta}$

Sequent Calculus for FOL at One Glance

	left side, antecedent	right side, succedent
$\forall$	$\dfrac{\Gamma, \forall\tau\, x;\ \phi,\ [x/t']\,\phi \Longrightarrow \Delta}{\Gamma, \forall\tau\, x;\ \phi \Longrightarrow \Delta}$	$\dfrac{\Gamma \Longrightarrow [x/c]\,\phi, \Delta}{\Gamma \Longrightarrow \forall\tau\, x;\ \phi, \Delta}$
$\exists$	$\dfrac{\Gamma, [x/c]\,\phi \Longrightarrow \Delta}{\Gamma, \exists\tau\, x;\ \phi \Longrightarrow \Delta}$	$\dfrac{\Gamma \Longrightarrow [x/t']\,\phi,\ \exists\tau\, x;\ \phi, \Delta}{\Gamma \Longrightarrow \exists\tau\, x;\ \phi, \Delta}$
$\doteq$	$\dfrac{\Gamma, t \doteq t' \Longrightarrow [t/t']\,\phi, \Delta}{\Gamma, t \doteq t' \Longrightarrow \phi, \Delta}$ (+ application rule on left side)	$\dfrac{}{\Gamma \Longrightarrow t \doteq t, \Delta}$

- $[t/t']\,\phi$ is result of replacing each occurrence of t in ϕ with t'
- t, t' arbitrary variable-free terms of type τ
- c new constant of type τ (occurs not on current proof branch)
- Equations can be reversed by symmetry

Logique dynamique avec substitutions explicites

Règles de déduction (1/2)

$$\frac{C \Longrightarrow \{\mathcal{U}, x := e\}\ F, D}{C \Longrightarrow \{\mathcal{U}\}\ [x = e;]\ F, D} \qquad (\textit{affectation})$$

$$\frac{C \Longrightarrow \{\mathcal{U}\}\ [p]\,[q]\ F, D}{C \Longrightarrow \{\mathcal{U}\}\ [\{p\ q\}]\ F, D} \qquad (\textit{bloc})$$

où $\{\ p_1\ \ldots p_n\ \}$ abrège $\{\ p_1\ \{\ \ldots\ \{\ p_{n-1}\ p_n\ \}\ \ldots\ \}$

$$\frac{\begin{array}{c} C \Longrightarrow \{\mathcal{U}\}\ (b\ \texttt{->}\ [p]\ F), D \\ C \Longrightarrow \{\mathcal{U}\}\ (\texttt{(!}\ b\texttt{)}\ \texttt{->}\ F), D \end{array}}{C \Longrightarrow \{\mathcal{U}\}\ [\texttt{if}\ \ \texttt{(}b\texttt{)}\ p]\ F, D} \qquad (\textit{si-alors})$$

$$\frac{\begin{array}{c} C \Longrightarrow \{\mathcal{U}\}\ (b\ \texttt{->}\ [p]F), D \\ C \Longrightarrow \{\mathcal{U}\}\ (\texttt{(!}b\texttt{)}\ \texttt{->}\ [q]\ F), D \end{array}}{C \Longrightarrow \{\mathcal{U}\}[\texttt{if}\ \ \texttt{(}b\texttt{)}\ \ p\ \texttt{else}\ \ q]\ F, D} \qquad (\textit{si-alors-sinon})$$

Logique dynamique avec substitutions explicites

Règles de déduction (2/2)

$$\frac{\begin{array}{c} C \Longrightarrow \{\mathcal{U}\}\ I, D \\ C \Longrightarrow \{\mathcal{U}, \mathcal{V}\}\ (\texttt{(}I\ \texttt{\&}\ b\texttt{)}\ \texttt{->}\ [p]\ I), D \\ C \Longrightarrow \{\mathcal{U}, \mathcal{V}\}\ (\texttt{(}I\ \texttt{\&}\ \texttt{!}\ b\texttt{)}\ \texttt{->}\ F), D \end{array}}{C \Longrightarrow \{\mathcal{U}\}\ [\texttt{while}\ \ \texttt{(}b\texttt{)}\ \ p]\ F, D} \qquad (\textit{tant-que})$$

où la substitution $\mathcal{V}$ rafraîchit (remplace par des variables fraiches) toutes les variables que le programme p peut modifier

- Trouver l'invariant de boucle I